

setup_brk:

A função `setup_brk` tem como objetivo inicializar nossa heap. Para isso, fazemos uma chamada de sistema `sys_brk` passando o argumento 0, o que retorna o valor atual do break do processo. Esse valor é armazenado tanto em `base_brk` quanto em `brk`. Assim, definimos o ponto inicial da nossa heap e o ponteiro atual do topo dela. Essa função é chamada uma única vez antes de qualquer operação de alocação.

dismiss_brk:

A função `dismiss_brk` basicamente restaura o estado original da heap. Ela faz uma chamada para `sys_brk` passando como argumento o valor de `base_brk`, que é o primeiro valor retornado no `setup`. Ou seja, ela "descarta" todo o espaço alocado após o `setup`, retornando o break para o seu ponto inicial. Após isso, atualiza também a variável `brk` para manter a consistência com o estado real do break no sistema.

memory_alloc:

Nosso código basicamente em um primeiro momento percorrerá os blocos já alocados (se houver) e verificará se possui blocos disponíveis com tamanho suficiente. Ainda dentro dessa lógica, ele verifica o maior tamanho entre os disponíveis (Worst-Fit). Caso encontre um bloco disponível ele aloca e faz uma terceira verificação, se esse bloco possui o tamanho do `memory_alloc` mais 10 bytes, sendo 9 de metadados e 1 com o tamanho mínimo de um bloco, para dividir esse bloco encontrado em dois. Caso seja possível ele faz o split, caso não seja ele aloca no bloco com tamanho integral. Além disso, caso ele não ache um bloco previamente liberado com tamanho suficiente, ele vai até o final da heap, calcula o novo valor do `brk`, faz a chamada `sys_brk` para estender a heap, escreve o byte de uso e os 8 bytes de tamanho do novo bloco criado, e já retorna o ponteiro para os dados desse bloco recém-criado, exatamente do tamanho requisitado pelo usuário.

memory_free:

A função `memory_free` é bastante direta: ela recebe o ponteiro para o começo dos dados do bloco que se deseja liberar. Como sabemos que todo bloco possui 9 bytes de metadados antes da área de dados (1 byte de uso + 8 bytes de tamanho), basta subtrair 9 para chegar ao início dos metadados. Nesse endereço, mudamos o byte de uso para 0, indicando que o bloco agora está livre. O restante da estrutura permanece intacto, preservando o tamanho original do bloco. Não há junção automática (coalescência) com blocos adjacentes, portanto essa função apenas marca o bloco como livre.

Script: Testamos os seguintes casos: Alocar no final da heap, alocar em bloco previamente liberado sem split, alocar em bloco previamente liberado com split e alguns casos pra testar worst fit.